Chapter 20

# Cost, recursion, links and trees

*count the steps, and the seconds take care of themselves*

A faster computer buys you a constant factor. A better algorithm changes the **shape** of the cost. Here is one question, "does this list contain a duplicate?", answered three ways, with the steps each one takes counted rather than timed:

```
import random
from functools import cmp_to_key

def by_pairs(xs):                   # compare every pair
    steps = 0
    for i, a in enumerate(xs):
        for b in xs[i + 1:]:
            steps += 1
            if a == b:
                return steps
    return steps

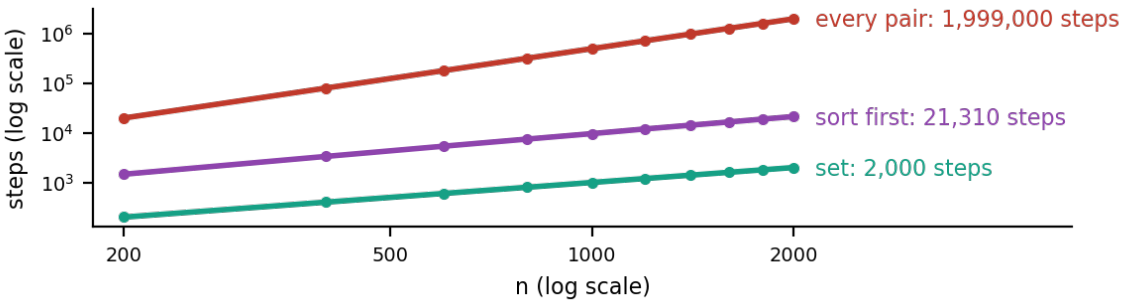
def by_sorting(xs):                 # sort, then compare neighbours
    steps = 0
    def cmp(a, b):
        nonlocal steps
        steps += 1
        return (a > b) - (a < b)
    sorted(xs, key=cmp_to_key(cmp))
    return steps + len(xs) - 1

def by_set(xs):                     # one hash lookup per item
    return len(xs)

rng = random.Random(1)
for n in (250, 500, 1000, 2000):
    xs = rng.sample(range(10 * n), n)          # no duplicates: the worst case
    print(f"{n:>5} {by_pairs(xs):>10,} {by_sorting(xs):>7,} {by_set(xs):>6,}")
```

```
  250     31,125   1,911    250
  500    124,750   4,339    500
 1000    499,500   9,653  1,000
 2000  1,999,000  21,307  2,000
```

*line 17: counts each comparison Timsort makes*




Double n and the pair-checker does **four times** the work, sorting a bit more than twice, the set exactly twice. On log scales that shows up as the **slope**: 2 for $n^2$, 1 for n. That ratio, not the seconds on one machine, is what Big-O describes.

## Big-O: keep the term that wins

Big-O names how the steps grow as n grows, ignoring constant factors and smaller terms. `3n² + 50n + 7` is O($n^2$) because for large n the $n^2$ term is all that matters:

| class | example | n = 10 | n = 1,000 | n = 1,000,000 |
|---|---|---|---|---|
| `O(1)` | dict lookup | `1` | `1` | `1` |
| `O(log n)` | binary search | `3` | `10` | `20` |
| `O(n)` | one loop | `10` | `1,000` | `1,000,000` |
| `O(n log n)` | sorting | `33` | `9,966` | `≈ 10^7` |
| `O(n²)` | every pair | `100` | `1,000,000` | `≈ 10^12` |
| `O(2ⁿ)` | every subset | `1,024` | `≈ 10^301` | `≈ 10^301,030` |

**RULE OF THUMB**

Steps in sequence **add** (keep the biggest). Loops **multiply**: a loop of n around an O(n) body is O($n^2$). A **call counts with its own cost**: `x in some_list` inside a loop is a hidden inner loop. Halving the problem each step gives a log n.

### What Python's own operations cost

| structure | O(1) | O(n) or worse |
|---|---|---|
| `list` | `xs[i]`, `xs[i] = v`, `append`, `pop()`, `len` | `insert(0, v)`, `pop(0)`, `x in xs`, `remove`, `xs[a:b]` copies, `sort` O(n log n) |
| `dict` / `set` | `d[k]`, `k in d`, `d[k] = v`, `del d[k]` (on average) | iterating; a worst case of O(n) when many keys collide |
| `deque` | `append`, `appendleft`, `pop`, `popleft` | `dq[i]` near the middle |
| `str` | `s[i]`, `len(s)` | `s + t` copies both; `+=` in a loop can be O($n^2$): use `"".join` |

Measure to check a claim: double n and look at the ratio of the times. About 2 means linear, about 4 means quadratic:

```
import timeit

for fn in (by_pairs, by_set):
    times = []
    for n in (500, 1000, 2000):
        xs = list(range(n))
        times.append(min(timeit.repeat(lambda: fn(xs), number=1, repeat=3)))
    ratios = [f"x{b / a:.1f}" for a, b in zip(times, times[1:])]
    print(f"{fn.__name__:9} each doubling: {' '.join(ratios)}")
```

```
by_pairs  each doubling: x4.2 x4.0
by_set    each doubling: x0.7 x0.9
```

*line 7: best of 3 runs*

**HONEST NOTE**

Big-O hides constants, and constants matter at small n: `sorted()` on 20 items beats any clever structure. It also hides the machine: memory locality (Ch 18) and C-level loops can make an O(n) built-in faster than an O(log n) pure-Python one for every n you'll actually meet. Use Big-O to rule out designs, then measure (Ch 17).

## Amortised: why append is O(1) on average

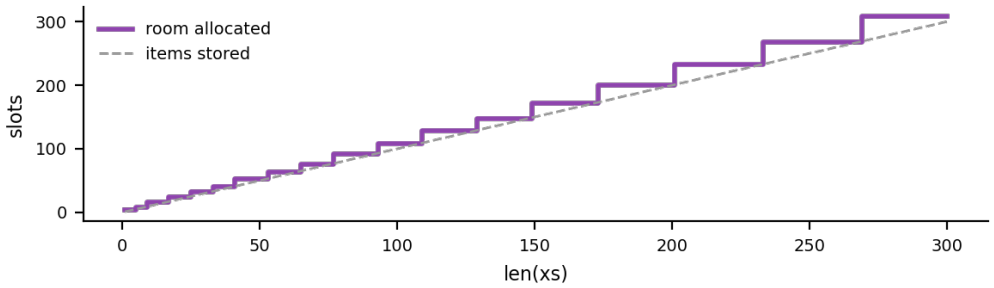
A list is an array of pointers with spare room at the end. When the room runs out, CPython allocates a bigger block and copies everything across: an O(n) step. It doesn't happen often, because each new block is about 1/8 larger than needed plus a little:

```
import sys

xs, last = [], sys.getsizeof([])
for i in range(40):
    xs.append(i)
    size = sys.getsizeof(xs)
    if size != last:                        # a new, bigger block was allocated
        room = (size - sys.getsizeof([])) // 8
        print(f"len {len(xs):>2}: room for {room} items")
        last = size
```

```
len  1: room for 4 items
len  5: room for 8 items
len  9: room for 16 items
len 17: room for 24 items
len 25: room for 32 items
len 33: room for 40 items
```

*line 8: 8 bytes per pointer*




**KEY IDEA**

**Amortised** cost spreads the rare expensive step over the many cheap ones. Because the block grows by a fixed **fraction**, the copying adds up to a constant number of moves per append, however long the list gets. Growing by a fixed **amount** instead (say +10 each time) would make append O(n) on average.

## Recursion: a function that trusts itself

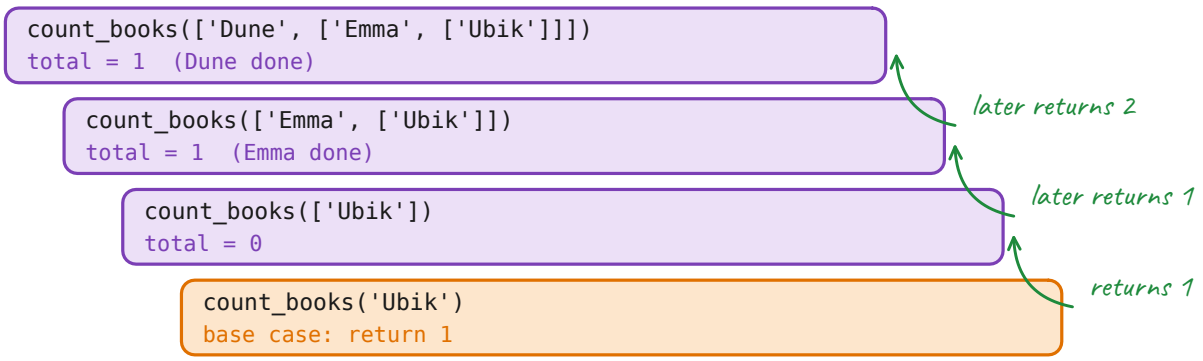
A recursive function solves a problem by solving **smaller** copies of the same problem, and stops at a **base case** small enough to answer directly. Nested data is the natural fit: a shelf holds books and boxes, and a box is a smaller shelf.

```
def count_books(shelf):
    if isinstance(shelf, str):          # base case: a single book
        return 1
    total = 0
    for item in shelf:                  # each item is a smaller shelf
        total += count_books(item)
    return total

print(count_books(["Dune", ["Emma", ["Ubik"]]]))
```

```
3
```

**the call stack at its deepest (each call has its own frame and its own total)**



**RULE OF THUMB**

Write recursion in three checks: (1) the base case returns without recursing, (2) every recursive call gets a **strictly smaller** input, (3) assuming the smaller calls are right, this call combines them correctly. That last step is proof by induction (13.5 Correctness through invariants and induction).

Each call is a new **frame** on the call stack (Ch 2), holding its own local variables, so the four `total`s never interfere. Python limits the stack's depth, and has no tail-call optimisation, so deep data needs a loop with an explicit stack instead:

```python
import sys

def count_books_loop(shelf):
    stack, total = [shelf], 0
    while stack:
        item = stack.pop()
        if isinstance(item, str):
            total += 1
        else:
            stack.extend(item)          # look inside this box later
    return total

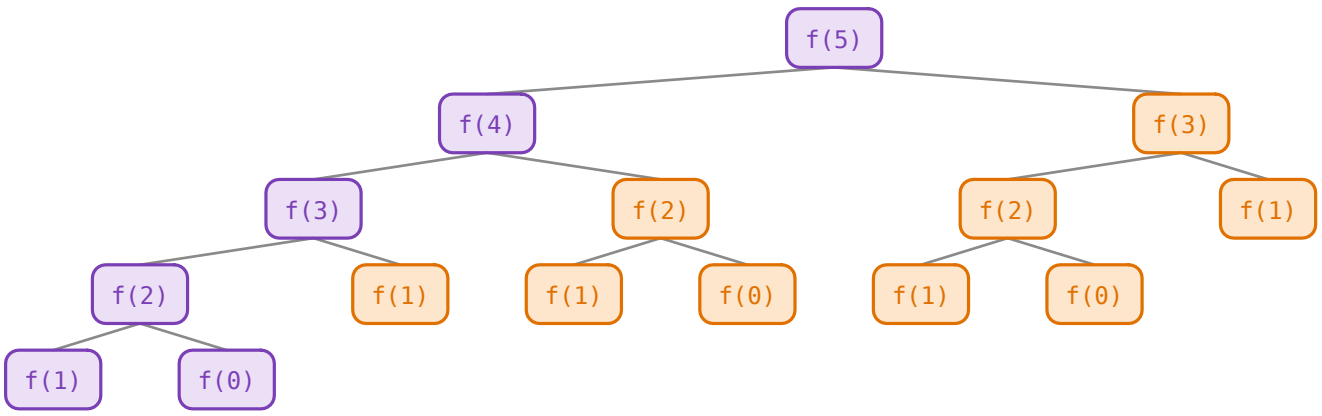
deep = "Dune"
for _ in range(5000):
    deep = [deep]                       # a book inside 5000 boxes
print(sys.getrecursionlimit(), count_books_loop(deep))
try:
    count_books(deep)
except RecursionError as e:
    print("RecursionError:", e)
```

← *the stack is now a list we own*

```
1000 1
RecursionError: maximum recursion depth exceeded
```

## Memoisation: never solve the same problem twice

The textbook Fibonacci definition calls itself twice, and the two calls overlap. Drawn out, `fib(5)` recomputes the same values again and again (orange):



```python
from functools import cache

calls = 0
def fib(n):
    global calls
    calls += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(20), calls)                    # exponential: about 1.6^n calls

@cache                                   # remember every answer by its argument
def fast(n):
    return n if n < 2 else fast(n - 1) + fast(n - 2)

print(fast(20), fast.cache_info())
print(len(str(fast(500))), "digits, instantly")
```

```
6765 21891
6765 CacheInfo(hits=18, misses=21, maxsize=None, currsize=21)
105 digits, instantly
```

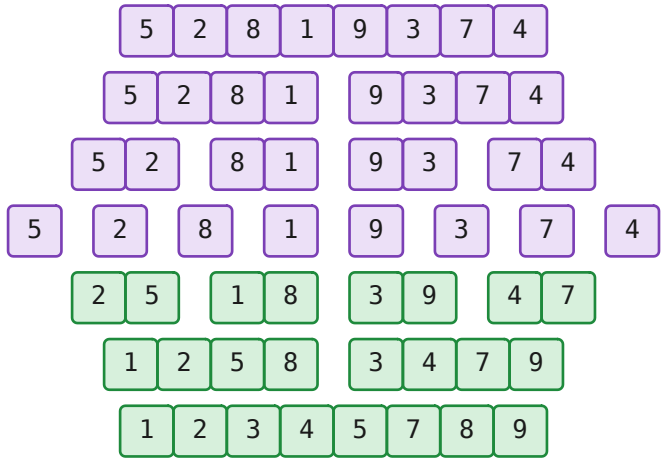
*line 7: 9 of fib(5)'s 15 calls are repeats*

**KEY IDEA**

`@cache` (or `@lru_cache(maxsize=N)` to bound memory) stores results in a dict keyed by the arguments, so the arguments must be hashable. Overlapping subproblems plus caching is the core of **dynamic programming**, Ch 21.

# Divide and conquer: merge sort

Split the list in half, sort each half (recursively), then **merge** two sorted lists by repeatedly taking the smaller front item. Splitting takes $\log_2 n$ levels, and every level merges all n items:




*split: halve until each piece has one item ($\log_2 8 = 3$ levels)*

*merge: each level walks all 8 items once, so n × log n*

```python
def merge_sort(xs):
    if len(xs) <= 1:
        return xs
    mid = len(xs) // 2
    left, right = merge_sort(xs[:mid]), merge_sort(xs[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

print(merge_sort([5, 2, 8, 1, 9, 3, 7, 4]))
loans = [("ana", 3), ("ben", 1), ("cy", 3), ("di", 1)]
print(sorted(loans, key=lambda p: p[1]))       # stable: ana stays before cy
```

```
[1, 2, 3, 4, 5, 7, 8, 9]
[('ben', 1), ('di', 1), ('ana', 3), ('cy', 3)]
```

*line 8: <= keeps equal items in order*

*line 12: one side may have leftovers*

**WHY**

Python's `sorted` is **Timsort**: merge sort that first finds runs already in order, so nearly-sorted data sorts in close to O(n). Like this merge sort it is **stable**: equal keys keep their order, which lets you sort by one key and then another. No comparison sort can beat O(n log n) in the worst case (13.8 Comparison sorting).

# Invariants: binary search you can trust

Binary search is famously easy to get almost right. The cure is to write down the **invariant**, a fact that is true before every loop step, and check that each line keeps it true:

```
1  def first_at_least(xs, target):
2      lo, hi = 0, len(xs)          # invariant: the answer is in range(lo, hi + 1)
3      while lo < hi:
4          mid = (lo + hi) // 2
5          if xs[mid] < target:
6              lo = mid + 1         # xs[mid] and everything left of it are too small
7          else:
8              hi = mid             # xs[mid] might be the answer: keep it
9      return lo                    # lo == hi: the range has one candidate left
10
11 ids = [3, 8, 15, 21, 27, 34, 40, 52, 66, 71]
12 print(first_at_least(ids, 40), first_at_least(ids, 41), first_at_least(ids, 99))
```

```
6 7 10
```

*line 2: hi = len(xs): 'past the end' is a valid answer*

*line 4: the range shrinks every step*

| lo | hi | mid | xs[mid] | step |
|---|---|---|---|---|
| 0 | 10 | 5 | 34 | 34 < 40: lo = 6 |
| 6 | 10 | 8 | 66 | 66 ≥ 40: hi = 8 |
| 6 | 8 | 7 | 52 | 52 ≥ 40: hi = 7 |
| 6 | 7 | 6 | 40 | 40 ≥ 40: hi = 6 |
| 6 | 6 | | | stop: answer 6 |

**TRAP**

Mixing conventions breaks it: `hi = len(xs) - 1` with `while lo < hi` misses the last item; `hi = mid - 1` throws away a possible answer; `lo = mid` (without + 1) loops forever when `hi == lo + 1`. Pick one invariant and derive every line from it. Or just use `bisect_left`, which is exactly this function (Ch 10).

# Linked lists from scratch

A linked list stores each item in its own small object, a **node**, that holds the value and a reference to the next node. There's no block to grow and nothing to shift: adding at the front is always O(1). Finding the 500th item means following 500 references.

```python
class Node:
    __slots__ = ("value", "next")            # no per-node __dict__: smaller nodes
    def __init__(self, value, next=None):
        self.value, self.next = value, next

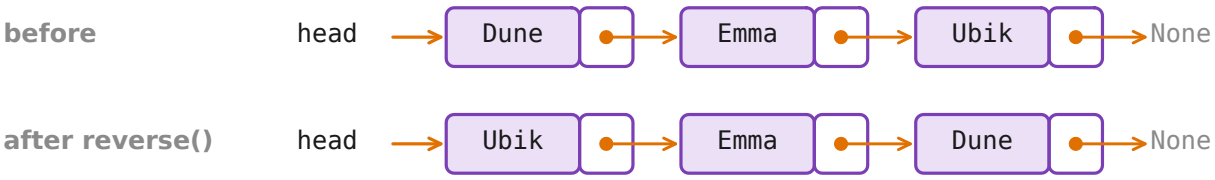
def from_items(items):
    head = None
    for v in reversed(items):
        head = Node(v, head)                 # O(1) push at the front
    return head

def items(node):
    while node is not None:
        yield node.value
        node = node.next

def reverse(node):
    prev = None
    while node is not None:
        nxt = node.next                      # remember the rest
        node.next = prev                     # turn this arrow around
        prev, node = node, nxt
    return prev

head = from_items(["Dune", "Emma", "Ubik"])
print(list(items(head)))
head = reverse(head)
print(list(items(head)))
```

```
['Dune', 'Emma', 'Ubik']
['Ubik', 'Emma', 'Dune']
```




| operation | Python list (array) | linked list |
|---|---|---|
| get item i | O(1): address arithmetic | **O(n)**: follow i references |
| add/remove at the front | **O(n)**: shift everything | O(1): one new node |
| insert after a node you hold | **O(n)** | O(1): relink two references |
| memory per item | 8 bytes (one pointer) | a whole node object (48 bytes, `sys.getsizeof`) |

**HONEST NOTE**

In Python you'll rarely write one: `list` and `deque` (which is itself a doubly linked list of 64-item blocks) are faster in practice. Linked structures matter because **trees and graphs are made of the same nodes and references**, and because in C and Rust you build them by hand. An LRU cache (practice 20.7) is the classic case where they win.

## Binary search trees

Give each node **two** references and keep one rule: everything in the left subtree is smaller, everything in the right subtree is larger. Then searching is binary search on a structure you can insert into, and an in-order walk visits the keys sorted:

```python
from dataclasses import dataclass

@dataclass(slots=True)
class TreeNode:
    key: int
    left: "TreeNode | None" = None
    right: "TreeNode | None" = None

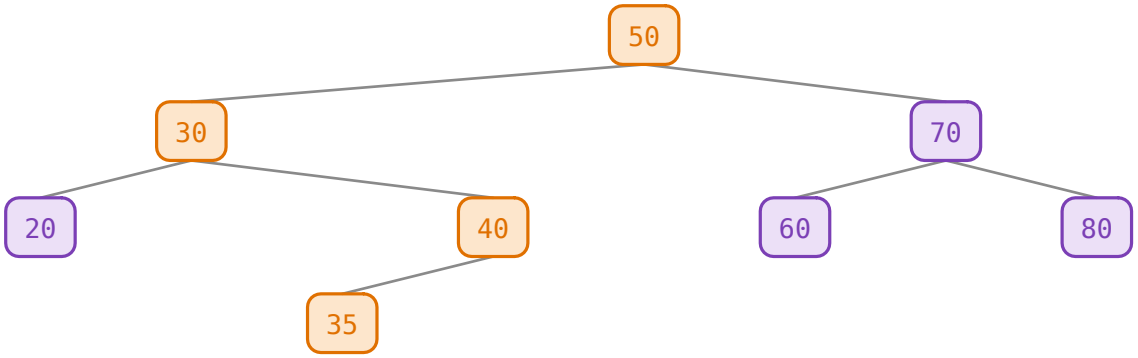
def insert(node, key):
    if node is None:
        return TreeNode(key)
    if key < node.key:
        node.left = insert(node.left, key)
    elif key > node.key:
        node.right = insert(node.right, key)
    return node                                # duplicates are ignored

def contains(node, key):
    while node is not None:                    # searching needs no recursion
        if key == node.key:
            return True
        node = node.left if key < node.key else node.right
    return False

def inorder(node):
    if node is not None:
        yield from inorder(node.left)
        yield node.key
        yield from inorder(node.right)

root = None
for k in [50, 30, 70, 20, 40, 60, 80, 35]:
    root = insert(root, k)
print(list(inorder(root)), contains(root, 35), contains(root, 36))
```

```
[20, 30, 35, 40, 50, 60, 70, 80] True False
```




Searching for 35 visits only the orange path: 50, 30, 40, 35. Each step discards a whole subtree, so a **balanced** tree of n keys needs about $\log_2 n$ steps. The four classic walks visit the same nodes in different orders:

```python
from collections import deque

def preorder(n):
    if n:
        yield n.key; yield from preorder(n.left); yield from preorder(n.right)

def postorder(n):
    if n:
        yield from postorder(n.left); yield from postorder(n.right); yield n.key

def level_order(root):
    queue = deque([root])
    while queue:
        n = queue.popleft()
        if n:
            yield n.key
            queue.extend((n.left, n.right))

print(list(preorder(root)), list(postorder(root)), list(level_order(root)), sep="\n")
```

```
[50, 30, 20, 40, 35, 70, 60, 80]
[20, 35, 40, 30, 60, 80, 70, 50]
[50, 30, 70, 20, 40, 60, 80, 35]
```

| walk | order | typical use |
|---|---|---|
| in-order | left, node, right | keys in sorted order |
| pre-order | node, then children | copy or print a tree (parents before children) |
| post-order | children, then node | delete or free a tree; sizes and heights from the bottom up |
| level order | row by row (a queue) | shortest path in steps: BFS, Ch 21 |

### The catch: insertion order decides the shape

```python
import random

def height(node):
    return 0 if node is None else 1 + max(height(node.left), height(node.right))

keys = list(range(500))
shuffled = random.Random(20).sample(keys, len(keys))
for name, order in [("random order", shuffled), ("sorted order", keys)]:
    root = None
    for k in order:
        root = insert(root, k)
    print(f"{name}: height {height(root)} for {len(keys)} keys")
```

```
random order: height 24 for 500 keys
sorted order: height 500 for 500 keys
```

*line 8: same keys, two insertion orders*

**HONEST NOTE**

Sorted input turns the tree into a linked list: O(n) per search. Real libraries use **self-balancing** trees (red-black, AVL) or **B-trees**, which is what SQLite indexes are (Ch 12). Python's standard library has none: use a `dict`/`set` for lookups, `bisect` on a sorted list for ranges, `heapq` for priorities, or the `sortedcontainers` package (Ch 19).

## ▢ Practise, then stamp

**NOW PRACTISE (practice/ folder, no installs needed)**

Run `python -m practice check 20.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 20.1` shows other valid solutions and their trade-offs.

20.1 measure and explain Members on both lists, fast
20.2 build from a spec Flatten anything, however deep
20.3 fix the bug Binary search, by the invariant
20.4 build from a spec A linked list with a tail
20.5 build from a spec A binary search tree that survives sorted input
20.6 refactor, tests stay green Count inversions in O(n log n)
20.7 mini-project An O(1) LRU cache

**PASSPORT STAMP** ask the same question of every new language (Appendix B)

**2. Memory model**

Linked structures are objects holding references to other objects; `__slots__` shrinks each node. A list is a growable array of pointers with spare room (amortised O(1) append).

**5. Functions and scope**

Recursion uses one stack frame per call; the default limit is 1000 and there is no tail-call optimisation, so deep data needs an explicit stack. `functools.cache` memoises.

**6. Data structures**

Know the costs: list O(1) at the end, O(n) at the front; dict/set O(1) on average; deque O(1) at both ends; heap O(log n); no balanced tree in the stdlib.

**IN OTHER LANGUAGES**

C: a node is a `struct` with a pointer field, and you `malloc`/`free` every node yourself. Java: references like Python, and `TreeMap` is a red-black tree in the standard library. C++: `std::map` is a balanced tree, `std::list` a linked list. Rust: `Option<Box<Node>>`, and ownership makes doubly linked lists famously hard; use `BTreeMap`. Scheme guarantees tail calls; Python and Java never eliminate them, and C compilers only sometimes do.

**DEEP DIVE (your vault)**

Vault notes: 13.1 A cost claim begins with a model · 13.2 Asymptotic notation · 13.3 Common growth classes · 13.4 Amortised analysis · 13.5 Correctness through invariants and induction · 13.7 Binary search · 13.8 Comparison sorting · 13.9 Python sorting and Timsort · 13.10 Linked lists built from scratch · 13.12 Binary search trees.